

PART III — BUILDING THE RETRIEVAL PIPELINE

Chapter 14

Prompt Engineering for RAG

“The instruction closest to the point of generation is the instruction the model actually obeys.”

Chapter 14 — Prompt Engineering for RAG

Memora's ``prompts.py`` holds more than a dozen distinct prompt families — draft generation, answer judging, chunk merging, redundancy scanning, compression, distillation — and none of them arrived at their current wording on a first attempt. ADR-012, one of the project's own architecture decisions, exists entirely because a prompt's *structure* (not its content) was silently causing tool calls to fire in the wrong order. Prompt engineering here was never decorative wording; it was iterated the same way code is — hypothesis, observed failure, targeted fix, regression note.

14.1 Anatomy of a good prompt — role, instruction, context, constraints

Definition — System prompt

The portion of a prompt that establishes persona, permissions, and standing rules for the model's behavior across a request, as distinct from the context (the evidence for this specific question) and the immediate instruction (what to do with it right now).

Memora's agent-facing prompt is split into exactly these parts, and the split is not cosmetic. ``_ROLE_AND_RULES`` establishes persona and hard limits; the retrieved context and any prior feedback are injected in the middle; ``_PROCESS_INSTRUCTIONS`` — the actual step-by-step task — is appended last, immediately before the user's message.

The anatomy of a grounded prompt

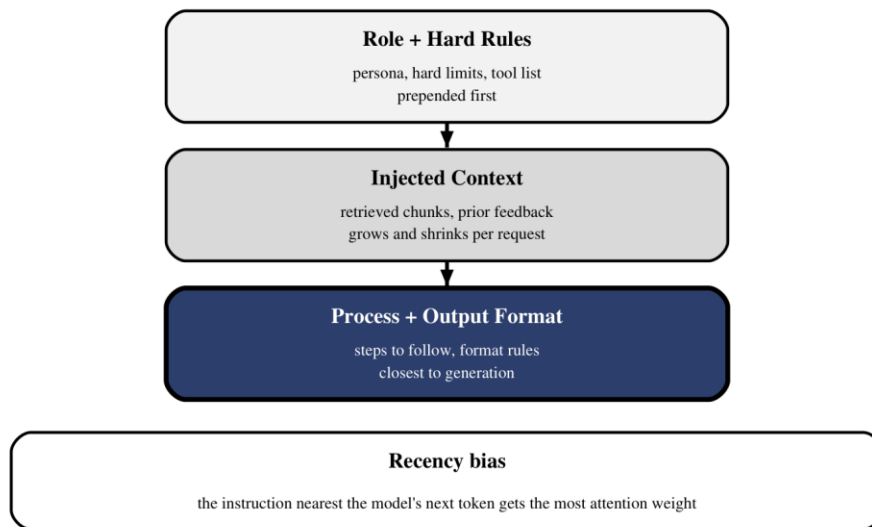


Figure 14.1 — Role and rules come first for stability; process instructions come last for attention.

That ordering in Figure 14.1 is itself a finding, not a convention borrowed from a style guide. Section 14.9 traces the exact failure that produced it: as injected context grew, instructions buried in the middle of the prompt were being followed less reliably.

Part	Memora's implementation	Purpose
Role	"You are a research assistant."	Frames every subsequent instruction
Rules	Hard limits — max retrievals, never fabricate	Standing constraints, not per-turn
Context	Retrieved chunks, thumbdown history	The evidence for this specific turn
Instruction	`_PROCESS_INSTRUCTIONS`, appended last	What to do, right now, with the above

14.2 Zero-shot, one-shot, few-shot prompting

Definition — Few-shot prompting

Including one or more worked input/output examples inside the prompt itself, so the model infers the expected pattern from demonstration rather than from a rule description alone.

Memora uses all three, deliberately matched task difficulty. `GROUNDING\_PROMPT` is zero-shot — the judgment ('does this answer address the query, grounded in these chunks?') is simple enough that a clear rule description is sufficient. The redundancy scanner is few-shot, because "do these two sentences express the same fact" is a judgment call where a rule alone leaves too much room for a smaller model to drift.

GOOD EXAMPLE:

Chunk 0: "ASD affects 1 in 36 children."

Chunk 1: "Approximately 1 in 36 children have autism."

These ARE redundant because they express the same fact.

NOT REDUNDANT — SAME TOPIC BUT DIFFERENT FACTS:

Chunk 0: "ASD patients may experience sensory overload."

Chunk 1: "Healthcare providers should reduce loud noises."

These are RELATED but NOT redundant.

Notice the negative example is doing as much work as the positive one — a rule stated in isolation ("redundant means same fact") is far more ambiguous than a rule shown failing on a plausible near-miss. Reach for few-shot examples exactly when a task has a plausible wrong answer that a rule alone won't rule out.

14.3 Chain-of-Thought and self-consistency

Definition — Chain-of-Thought (CoT) prompting

Instructing a model to produce intermediate reasoning steps before its final answer, on the premise that generating the steps improves the odds of reaching a correct conclusion, not merely explaining one.

CoT and strict output-format constraints (Section 14.12) pull in opposite directions, and Memora's judge prompts resolve the tension by picking a side deliberately: `'GROUNDING_PROMPT'` demands "Reply with EXACTLY one of these two lines" — no visible reasoning at all. That is not an oversight; a judge whose output must be parsed by code downstream cannot afford a model that reasons out loud before it commits to a verdict, however much that reasoning might have improved the verdict's quality.

Definition — Self-consistency

Sampling a model's chain-of-thought several times at nonzero temperature and taking the majority-vote answer, trading extra inference calls for a reasoning path that isn't sensitive to one unlucky sampling run.

Self-consistency is expensive in exactly the currency Chapter 13.7 showed is scarce — LLM calls — which is why Memora's design leans toward deterministic, temperature-0 judges instead: a judge that disagrees with itself between runs is a bigger problem than one that occasionally reasons imperfectly but consistently.

14.4 ReAct — reasoning and acting

Definition — ReAct

A prompting pattern that interleaves reasoning ("what do I need next") with acting ("call this tool to get it") in a loop, rather than asking a model to reason to a complete answer before taking any action.

Memora's own codebase never uses the word "ReAct," but Chapter 16 onward builds exactly this pattern: an agent loop that decides whether to retrieve, retrieves, evaluates what came back, and decides again — reasoning and acting interleaved rather than front-loaded. The prompt-engineering version of that idea is simpler than the architecture around it: give the model a reason to act between reasoning steps, rather than asking it to reason once, silently, all the way to a final answer.

14.5 The grounded-answer prompt template for RAG

Definition — Grounded answer

An answer whose claims are traceable to specific retrieved evidence, as distinct from a fluent answer that merely sounds plausible.

Every answer-generating prompt in this book, from `query.py`'s simplest version to the agent's full `\_ROLE\_AND\_RULES`, is a variation on the same four-part template: identity, question, evidence, instruction.

```
You are an expert assistant. Use the following retrieved documents to
answer the question.
```

```
Question: {query}
```

```
Context:
{context}
```

```
Provide a concise and accurate answer based on the above information.
```

The agent-facing version adds hard limits and tool descriptions around this same core, but the core itself never changes shape: identify the role, state the question, hand over the evidence, instruct the model to answer from it. Every technique in the rest of this chapter is a refinement of one of these four parts — never a fifth part bolted on.

14.6 Reducing hallucination through prompt constraints

"Answer ONLY from retrieved chunks — never from memory" and "Never fabricate facts" appear as explicit, standalone lines in `\_ROLE\_AND\_RULES` — not folded into a longer sentence, not implied by context. `GROUNDING\_PROMPT` then checks the same constraint from the other side, asking whether "the key claims in the answer are traceable to the retrieved chunks (not invented)."

Stating a constraint once and checking it once is weaker than stating it in the generation prompt and re-checking it in a separate judging prompt with a separate model call. A generation prompt's constraint shapes what the model is more likely to produce; a judging prompt's constraint catches what slipped through anyway. Chapter 20 builds the second half of that pair in full.

14.7 Citation-aware prompts and the [Source: filename] convention

Definition — Citation-aware prompt

A prompt that requires every factual claim to carry an inline, machine-parseable pointer back to the specific source it came from, so groundedness can be verified per-claim instead of trusted for the answer as a whole.

Memora's convention is exact and repeated verbatim across every prompt that produces cited text: ``[Source: filename]``, placed inline after the sentence it supports, never collected into a trailing list.

```
- Cite every source inline as [Source: filename] after the sentence it
  supports.
- If multiple sources support the same fact, list them all: [Source:
  a.pdf] [Source: b.pdf].
...
- Citations appear ONLY inline within sentences as [Source: filename].
  Never as a trailing list.
```

The "never as a trailing list" rule is not a style preference — a trailing citation list disconnects the source from the specific claim it supports, so a reader (or a downstream faithfulness checker) can no longer tell which of three sources backs which of five sentences. Inline placement keeps the pointer next to the claim it is a pointer for.

14.8 Controlling tone, length, and output format

``_PROCESS_INSTRUCTIONS``'s OUTPUT FORMAT block controls all three at once, in four short bullet lines: plain prose, no headings or markdown, a 400-word ceiling, and inline-only citations. Compare that to the judge and repair prompts elsewhere in the file, which demand the opposite of prose — a single JSON object, no markdown fences, first character mandated to be ``{`` or ``[``.

Output need	Format constraint used	Where
Human-readable answer	Plain prose, no markdown, word cap	<code>`_PROCESS_INSTRUCTIONS`</code>
Machine-parsed verdict	Exactly one of two literal lines	<code>`GROUNDING_PROMPT`</code>
Machine-parsed structure	Single JSON object, no fences, no prose	<code>`_JSON_REPAIR_PROMPT`</code> , <code>`_DC_SCAN_PROMPT`</code>

The format constraint is chosen by who reads the output next, not by what feels natural to write. A human reads prose; a parser needs a format it can call ``json.loads()`` on without a preprocessing step. Mixing the two — prose with an embedded JSON block, say — creates work for whichever consumer didn't get the format it needed.

14.9 Why word-count caps are honored as suggestions, not rules

The 400-word ceiling in `_PROCESS_INSTRUCTIONS` was not a stylistic preference from day one — it was added after a specific, observed failure. Under context dilution, the model entered a repetition-degeneration loop that produced an 11,133-character answer, vomiting citations in a cycle with no natural stopping point. The word cap, along with "no headings," "citations inline only," and "never as a trailing list," were added together as one OUTPUT FORMAT block specifically to close that failure mode.

Common pitfall — Treating a stated cap as an enforced one

Nothing downstream of generation in this pattern counts words and truncates the response — the 400-word instruction is exactly that, an instruction, honored to the degree an autoregressive model can honor a global property of text it is generating one token at a time without foresight of its own final length. Contrast this with Chapter 11's `score_threshold`, which is enforced in code after the fact regardless of what the retriever "intended." A prompt constraint shapes probability; a code constraint guarantees an outcome. Know which one you are relying on, and reach for the code-enforced version whenever the cost of violation is high enough to matter.

14.10 Debugging a bad prompt

Memora tracks every deliberate prompt revision in a dedicated `Prompt_Changes.txt` file, not scattered across commit messages — a small habit worth adopting directly: when a prompt changes because of an observed failure, write down what failed, what changed, and why, in one place a future edit can be checked against.

- Read the actual retrieved context the model saw, not what you assume was retrieved (Chapter 22's dry-run trace exists for exactly this).
- Isolate structure from content — move the failing instruction closer to the end of the prompt before rewriting its wording.
- Check whether the failure is a generation problem or a parsing problem; a well-grounded answer in the wrong format looks identical to a badly grounded one until you check which stage broke.
- Reproduce with the same model and temperature before concluding a fix worked — a single passing run proves little at temperature above 0.
- Prefer one small, explainable change per iteration; a rewrite that touches five instructions at once teaches you nothing about which one mattered.

14.11 The Conservative-Grounding Prompt Pattern

Definition — Conservative-Grounding Prompt Pattern

A reusable instruction template — applied identically across generation, drafting, and judging prompts — that forbids any claim, inference, or value not directly traceable to the supplied evidence, with an explicit instruction to prefer an empty or default result over a fabricated one.

This is not one prompt; it is a pattern repeated with the same backbone across roles that have nothing else in common. The answer generator is told to answer only from context. The merge judge is told a claim is fabricated unless a source supports it "verbatim or as a clear paraphrase." The value-verification prompt is told to "use the existing default... for genuinely missing fields" rather than invent one.

```
# Generation
Answer ONLY from retrieved chunks — never from memory.
Never fabricate facts.

# Judging (same backbone, applied to someone else's output)
For every factual claim in the MERGED CHUNK, ask: "Is this claim
supported by at least one SOURCE CHUNK above?"
If NO -> list it in "fabricated_claims".
```

The pattern's real value is consistency across roles: a project that only forbids fabrication at generation time has one gate; the same rule enforced identically at generation, drafting, and judging is three independent chances to catch the same failure mode, in the same vocabulary, checkable against each other.

14.12 Structure for reliable structured-output LLM calls

Every JSON-producing prompt in `prompts.py` — the redundancy scanner, the merge judge, the JSON repair model — follows the identical five-layer shape shown in Figure 14.2, in the identical order.

The structured-output prompt hierarchy

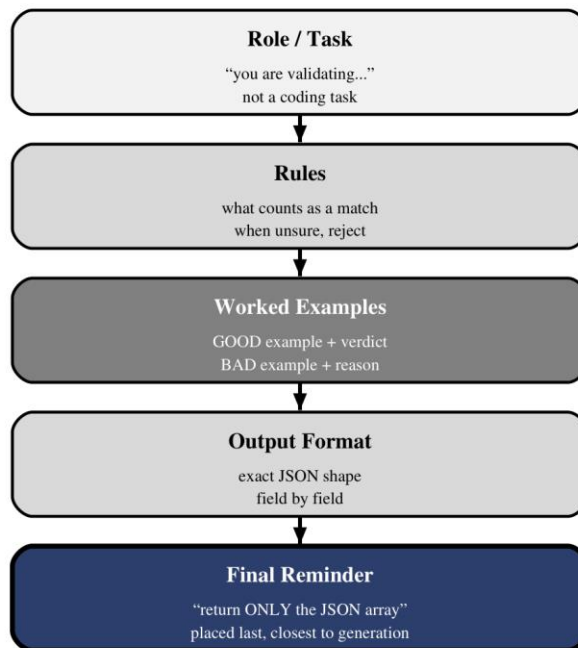


Figure 14.2 — The reminder that matters most for output compliance goes last, nearest the point of generation.

That ordering is the same recency-bias finding from Section 14.1 and ADR-012, applied to a different problem: a rule stated once at the top of a long prompt is exactly as vulnerable to being "forgotten" by the time generation starts as a process instruction buried in the middle of an agent's system prompt was.


```
Return ONLY a JSON object – no markdown, no prose, no code fences:
{{"compressed": "<retained content, or __IRRELEVANT__>", "dropped_count":
<int>, "reason": "<one sentence>"}}
```

...

```
The first character of your response must be '['.
Your response will be parsed directly using json.loads().
Invalid JSON will cause failure.
```

```
Return ONLY the JSON array.
```

The repeated, almost redundant-sounding final reminder — stating the format constraint a second time, immediately before generation begins — is not padding. It is the same lesson as the system prompt split, compressed into three lines instead of two files.

14.13 Advanced reasoning prompts — Tree of Thoughts, Step-Back, and Socratic prompting

Definition — Tree of Thoughts (ToT)

A reasoning strategy that explores several candidate reasoning branches in parallel, evaluates each, and continues only the most promising ones — trading a single linear chain-of-thought for a searchable tree of partial solutions.

None of these three techniques appear in Memora's own prompt library — they are heavier-weight tools than a RAG answer or a compression judgment typically needs, and they belong in this chapter as tools worth recognizing rather than ones this particular project reached for.

Technique	Core idea	Best fit
Tree of Thoughts	Explore and prune multiple reasoning branches	Problems with many plausible partial solutions
Step-Back prompting	Ask a general question before the specific one	Questions that need a principle before a detail
Socratic prompting	Have the model interrogate its own draft with questions	Self-review passes on a completed answer

A useful filter for all three: reach for them when a single forward pass of reasoning is demonstrably insufficient, not by default. Memora's own judges deliberately avoid open-ended reasoning (Section 14.3) precisely because their task — a bounded classification — does not need a searchable reasoning tree to get right.

14.14 Multi-stage prompting — prompt chaining and meta prompting

Definition — Prompt chaining

Splitting a task across two or more sequential LLM calls, where each call's output becomes the next call's input, instead of asking one prompt to do the entire task in one pass.

Memora's answer generation is a real, shipped example. An earlier design asked one call to produce the final answer directly from context. It was later split into `'generate_draft'` — produce a working draft from raw context — followed by `'generate_answer'`, which takes that draft as synthesis input and produces the answer returned to the user. The draft is not the answer; it is the first link in a two-call chain, and Chapter 20 covers the quality-gate bug that this exact split once introduced when the two stages' ordering wasn't reconciled with a judge sitting between them.

Definition — Meta prompting

A prompt whose job is to construct, evaluate, or repair another prompt's output, rather than to answer the user's original question directly.

The `'_JSON_REPAIR_PROMPT'` and `'_VALUE_VERIFY_PROMPT'` are both meta prompts in exactly this sense — neither one answers a user's question; each one exists to fix what an earlier call in the chain produced. A pipeline with enough prompt chaining tends to accumulate meta prompts almost by necessity, since more stages means more places a stage's output can arrive malformed.

14.15 Delimiter techniques

Definition — Delimiter

A visual or structural marker — capitalized labels, brackets, fenced blocks — that separates one part of a prompt (instructions, examples, evidence, user input) from another, so the model does not have to infer the boundary from prose alone.

Memora's delimiter of choice, used identically across every prompt in the file, is a capitalized label on its own line: `'USER QUERY:'`, `'RETRIEVED CHUNKS:'`, `'ANSWER TO EVALUATE:'`, `'PROPOSED GROUPS:'`. No XML tags, no triple backticks around plain text sections — just a consistent, unambiguous label immediately before each block.

```
USER QUERY:
{query}

RETRIEVED CHUNKS (the only allowed source of facts):
{context}

ANSWER TO EVALUATE:
{answer}
```

The label does two jobs at once: it tells the model where one section ends and the next begins, and its wording — "the only allowed source of facts" — smuggles a constraint into what looks like a plain section header. A delimiter is free real estate for reinforcing a rule the model needs to see again anyway.

14.16 Persona and constraint hybrids

"You are a research assistant" is a persona. "You are NOT writing software. You are NOT generating Python. You are NOT solving a coding task" — three negative constraints, stacked immediately after a persona line in the redundancy scanner — is something else: a persona paired with an explicit boundary on what that persona is not permitted to drift into.

Common pitfall — A structured-comparison task read as a coding task

Smaller, general-purpose models asked to compare sentences for redundancy will sometimes respond as if the task were "write a script that checks for redundancy" — a plausible-sounding but useless answer to a task that needed a direct judgment, not code. `\_DC\_SCAN\_PROMPT` and its sibling judge prompts address this by stating the negative constraint explicitly and early, immediately after the persona, rather than assuming the task framing alone rules it out.

The pairing generalizes: a persona alone describes who the model should sound like; a persona plus explicit negative constraints describes who the model should sound like and which nearby, tempting failure mode that persona is specifically not allowed to slide into. The second half is only worth writing once you have actually watched a model slide.

Every prompt in this chapter was tuned against something that broke — a recency-bias failure, a repetition loop, a model that wrote code instead of judging. That is the throughline worth keeping past this chapter: a prompt is not finished when it reads well, only when it has survived contact with the model's actual, imperfect behavior. Part IV picks up from here and asks a harder question than any single prompt can answer alone — not just how to phrase an instruction well, but how to build a system that decides, on its own, when to retrieve, when to stop, and when to admit it does not yet know enough.